

Dokumentacja Projektu: Ticketmaster Weather

Martyna Sadowska, Liliana Sirko

9 czerwca 2026

1 Informacje ogólne

1.1 Cel i opis projektu

Projekt **Ticketmaster Weather** to aplikacja internetowa, której głównym celem jest integracja informacji o nadchodzących wydarzeniach kulturalnych, sportowych i rozrywkowych z precyzyjnymi prognozami pogody dla lokalizacji tych wydarzeń. System rozwiązuje częsty problem uczestników imprez (zwłaszcza plenerowych i wyjazdowych), pozwalając na jednoczesne planowanie rozrywki i przygotowanie się na warunki atmosferyczne bez konieczności ręcznego sprawdzania danych w kilku różnych serwisach.

1.2 Główne założenia i funkcjonalności

Aplikacja posiada następujący zestaw funkcjonalności:

- **Wyszukiwanie wydarzeń:** Użytkownik ma możliwość wyszukiwania wydarzeń na podstawie nazwy miasta, promienia poszukiwań (w kilometrach) oraz zdefiniowanego przedziału czasowego (od konkretnej daty do konkretnej daty).
- **Zakres czasowy prognoz:** Z uwagi na ograniczenia modeli meteorologicznych oraz specyfikację Open-Meteo API, aplikacja obsługuje wyszukiwanie i precyzyjne prognozowanie pogody do 14 dni w przód. Wyszukiwanie dat z przeszłości i przyszłości dalszej niż 2 tygodnie nie jest obsługiwane (taka opcja jest wyłączona w UI).
- **Integracja i mapowanie danych:** System automatycznie odpytuje Ticketmaster API o szczegóły wydarzeń (ceny biletów, kategoria imprezy, adres), koduje ich lokalizacje, a następnie paruje je z prognozą pogody dla uzyskanych współrzędnych geograficznych.
- **Mechanizm cache:** Aby zminimalizować opóźnienia sieciowe i zredukować zużycie limitów zewnętrznych API, system asynchronicznie zapisuje przetworzone wyniki w relacyjnej bazie danych. Ponowne zapytania o wydarzenia o tych samych kryteriach są pobierane z bazy danych.

- **Interaktywna wizualizacja:** Wyniki zapytań (dane o wydarzeniu + prognoza) są prezentowane w formie znaczników na interaktywnej mapie. Po najechaniu na znacznik wyświetlane są szczegóły danego wydarzenia lub wydarzeń (w przypadku, gdy to samo wydarzenie odbywa się przez kilka dni). Po dodatkowym wybraniu daty na widżecie kalendarza użytkownik może zobaczyć informacje pogodowe oraz szczegóły wydarzeń dla wybranego dnia.

1.3 Zespół i podział obowiązków

Projekt został zrealizowany w dwuosobowym zespole. Większość planowania architektury aplikacji była wykonywana wspólnie, natomiast dalszy rozwój poszczególnych warstw aplikacji został podzielony następująco:

- **Martyna Sadowska:** opis architektury systemu na diagramach, frontend aplikacji, logi i debugowanie,
- **Liliana Sirko:** backend aplikacji, testy jednostkowe i wydajnościowe, dokumentacja.

2 Architektura systemu

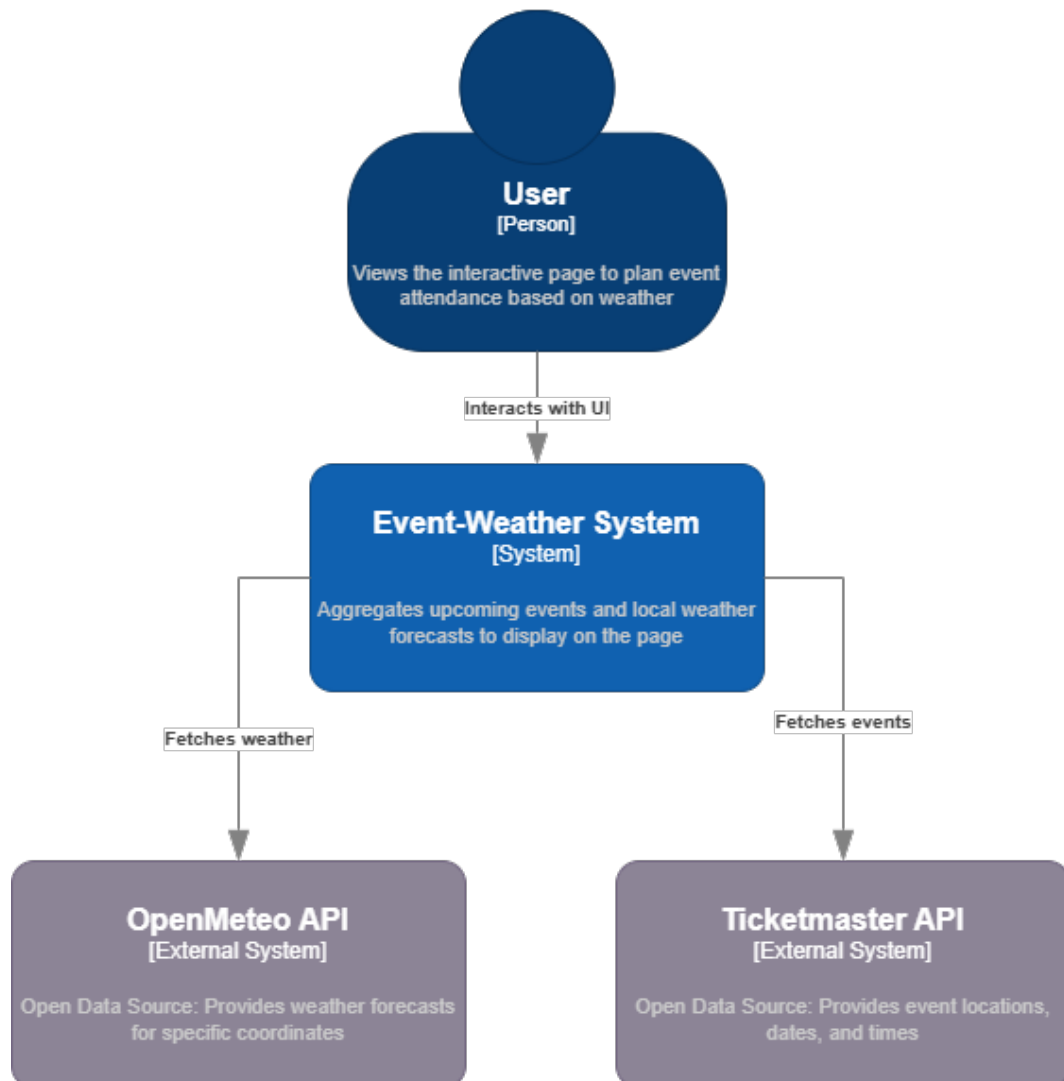
2.1 Wizualizacja Architektury

Do wizualizacji architektury aplikacji użyte zostały diagramy C4. Diagram C4 Context prezentujący interakcje użytkownika z systemem, widoczny jest na Rysunku 1. Diagram C4 Container prezentujący podział wewnętrznej architektury na kontenery umieszczony jest na Rysunku 2. Diagram C4 Component prezentujący wewnętrzną strukturę kontenera backendowego widoczny jest na Rysunku 3.

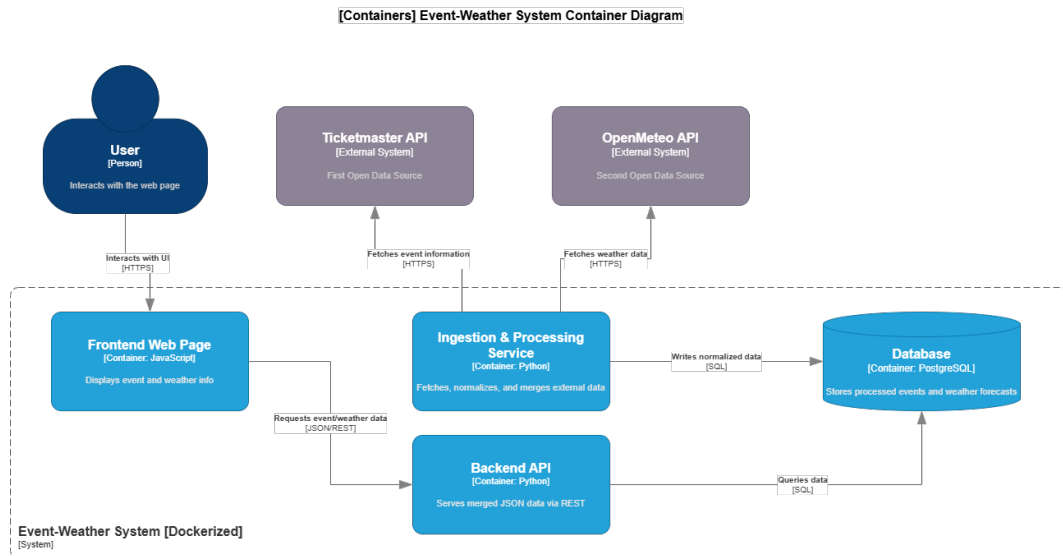
2.2 Opis Komponentów

- **Źródła danych:** Aplikacja pozyskuje i łączy dane z dwóch zewnętrznych interfejsów API. Ticketmaster API (zabezpieczone kluczem dostępu) dostarcza informacje o wydarzeniach kulturowych, cenach biletów i klasyfikacji. Open-Meteo API (otwarte API) jest wykorzystywane do pobierania prognoz pogody (temperatura, opady, wiatr) na podstawie współrzędnych i daty wydarzenia.
- **Moduł akwizycji (Data Ingestion Service):** Logika zawarta w pliku `app.ingestion`, odpowiadająca wyłącznie za komunikację sieciową z zewnętrznymi API. Wykonuje geokodowanie miast oraz pobiera surowe dane (JSON) z Ticketmaster i Open-Meteo. Moduł implementuje mechanizm cache.

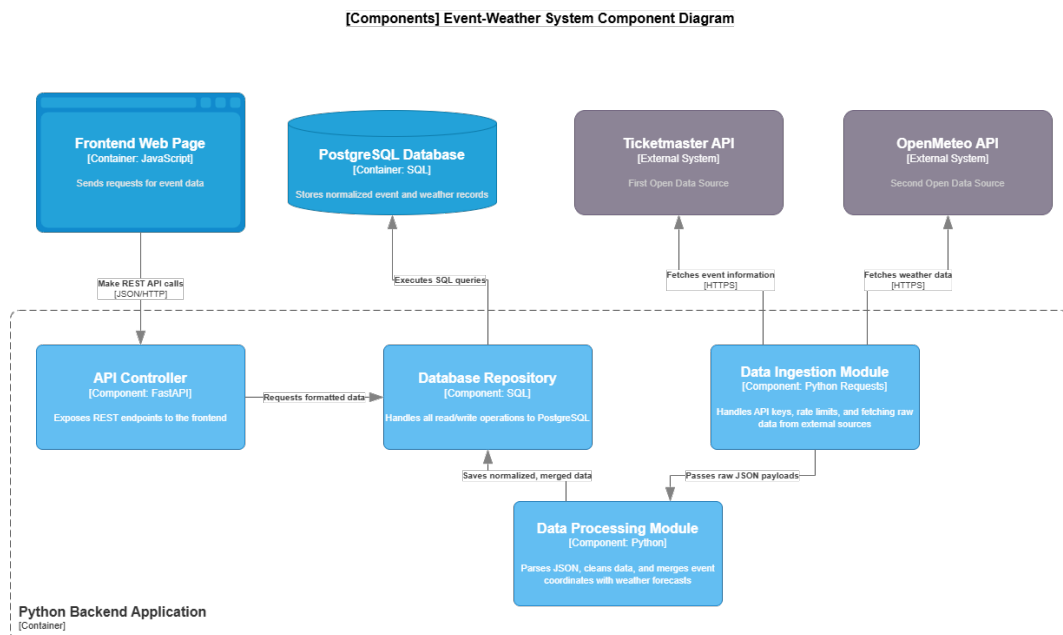
[System Context] Event-Weather System Context Diagram



Rysunek 1: C4 Context Diagram systemu prezentujący głównego aktora (użytkownika) oraz interakcje całego systemu Event-Weather z zewnętrznymi źródłami danych (Ticketmaster API i OpenWeather API).



Rysunek 2: C4 Container Diagram systemu prezentujący jego wewnętrzną architekturę z podziałem na główne wdrażalne kontenery, takie jak Frontend, Backend API, moduł akwizycji danych oraz bazę danych PostgreSQL.



Rysunek 3: C4 Component Diagram systemu prezentujący wewnętrzną strukturę kontenera backendowego, z wyszczególnieniem kluczowych komponentów aplikacji.

- **Moduł przetwarzania i parsowania danych:** Komponent odpowiedzialny za logikę biznesową. Parsuje surowe odpowiedzi z API, obsługuje potencjalne braki w danych (np. brakujące ceny lub adresy) poprzez mechanizmy bezpiecznego mapowania, a następnie łączy wydarzenia z odpowiadającą im w czasie prognozą pogody.
- **Warstwa bazy danych:** Moduł zarządzający relacyjną bazą PostgreSQL przy użyciu biblioteki SQLAlchemy ORM. Tabela `event_weather_records` przechowuje pełne informacje o wydarzeniach oraz warunkach pogodowych, pełniąc funkcję cache w celu minimalizacji odpytywań zewnętrznych API. Dane posiadają datę pobrania - wpisy starsze niż 24 godziny są usuwane z bazy danych.
- **Backend (API):** Aplikacja jest oparta na frameworku FastAPI. Udostępnia endpoint `/api/search`, waliduje parametry wejściowe od użytkownika i koordynuje pracę pozostałych modułów. Implementuje architekturę dependency injection do obsługi połączeń z bazą danych i zwraca wyniki w formacie JSON.
- **Frontend (WWW):** Interfejs użytkownika oparty na czystym HTML/JS i bibliotece Leaflet, komunikujący się z backendem (REST API) w celu interaktywnej prezentacji połączonych danych o wydarzeniach i prognozie pogody. Wyświetla interaktywną mapę i kalendarz wydarzeń wraz z informacjami o wydarzeniach i pogodzie. Pozwala na filtrowanie po datach oraz promieniu wyszukiwania.
- **Środowisko uruchomieniowe:** Aplikacja jest w pełni skonteneryzowana. Moduł backendowy działa na wydajnym serwerze asynchronicznym Uvicorn, frontend serwowany jest przez lekki serwer Nginx, a całość zarządzana jest przez Docker Compose z podziałem na izolowane środowiska (dev, test, prod).

3 Warstwowa architektura aplikacji

Aplikacja została zbudowana zgodnie z trójwarstwowym modelem architektury. Warstwy zdefiniowane są wprost w plikach docker-compose. Wydzielamy następujące warstwy:

- **Warstwa prezentacji:** Odpowiada za bezpośrednią interakcję z użytkownikiem. W strukturze projektu jest reprezentowana przez niezależny kontener frontendowy (oparty na serwerze Nginx), serwujący aplikację webową (HTML, CSS, JavaScript). Jej głównym zadaniem jest renderowanie formularza wyszukiwania oraz wizualizacja połączonych danych na interaktywnej mapie. Warstwa ta jest całkowicie odseparowana od logiki przetwarzania - nie posiada wiedzy o strukturze bazy danych ani mechanizmach zewnętrznych API, a komunikację z systemem realizuje

wyłącznie poprzez asynchroniczne zapytania HTTP do punktów końcowych REST API backendu.

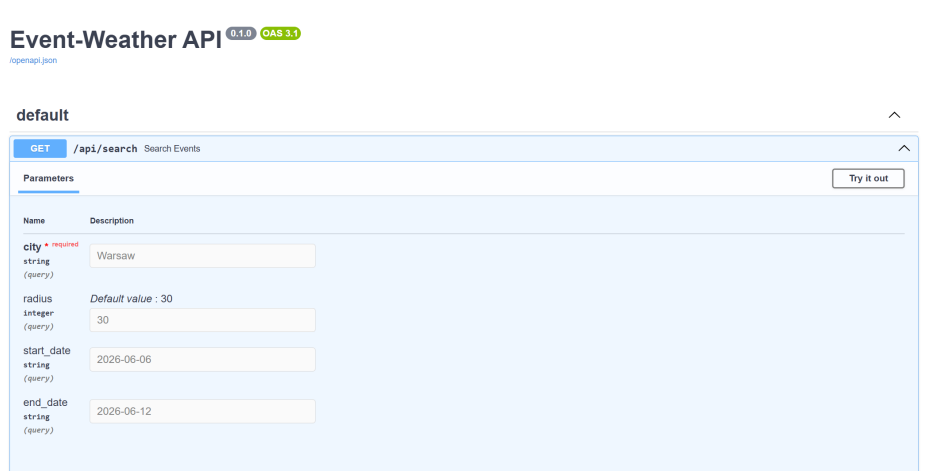
- **Warstwa logiki biznesowej:** Stanowi rdzeń obliczeniowy systemu i jest realizowana przez kontener backendowy oparty na frameworku FastAPI (Python). Warstwa ta odpowiada za walidację parametrów wejściowych, koordynację przepływu danych oraz implementację algorytmów integracji informacji. To tutaj zapada decyzja o wykorzystaniu lokalnego mechanizmu cache'owania - serwer weryfikuje dostępność danych w bazie, a w przypadku ich braku, zarządza sekwencyjnym odpytywaniem zewnętrznych serwisów (Ticketmaster API oraz Open-Meteo API), geokodowaniem współrzędnych i unifikacją wyników do ustandaryzowanego formatu JSON.
- **Warstwa dostępu do danych:** Jest odpowiedzialna za trwałe przechowywanie danych. W projekcie warstwę tą reprezentuje wyizolowany kontener z relacyjną bazą danych PostgreSQL. Backend komunikuje się z nią za pośrednictwem warstwy mapowania obiektowo-relacyjnego (ORM) SQLAlchemy. Baza danych przechowuje połączone rekordy historycznych i przyszłych zapytań (`EventWeatherRecord`), pełniąc kluczową rolę wydajnościową jako lokalny bufor (cache), eliminujący narzut sieciowy i ograniczenia limitów zapytań API.

4 Uzasadnienie wyboru technologii

- **Język programowania (Python + FastAPI):** Python oferuje najwyższą zwiezłość i elastyczność przy pracy z danymi. FastAPI został wybrany ze względu na natywną obsługę asynchroniczności, doskonałą wydajność i wbudowaną automatyczną dokumentację (Swagger).
- **Baza danych (PostgreSQL):** Wybór podyktowany wymogiem spójności typów danych (szczególnie restrykcyjnej obsługi dat i czasu w strefie UTC) oraz możliwością łatwej replikacji środowisk (Environment Parity) między rozwojem, testami a produkcją.
- **Narzędzia Orchestracji (Docker & Docker Compose):** Umożliwiają pełną izolację komponentów (osobne usługi dla backendu, bazy danych i frontendu). Pozwala to na zastosowanie podejścia multi-stage build (oddzielne obrazy dla środowiska developerskiego, testowego i produkcyjnego) oraz bezproblemowe zarządzanie sieciami (np. wyizolowana sieć `ticketmaster_test`).
- **Framework Testowy (Pytest):** Zastąpił wbudowany moduł `unit test` z uwagi na brak kodu "boilerplate" (możliwość pisania testów jako czystych funkcji) oraz potężny system modułowych "fixtures", który idealnie współgra ze wstrzykiwaniem zależności w FastAPI.

5 Dokumentacja API

Specyfikacja interfejsu programistycznego jest generowana automatycznie w standardzie OpenAPI za pomocą wbudowanych mechanizmów frameworka FastAPI. Interaktywna dokumentacja (Swagger UI) pozwala na testowanie endpointów na żywo oraz podgląd struktur danych wejściowych i wyjściowych. W środowisku lokalnym jest ona dostępna pod adresem <http://localhost:8000/docs>. Podgląd interfejsu zaprezentowano na Rysunku 4.



Rysunek 4: Interaktywna dokumentacja API wygenerowana za pomocą Swagger UI.

6 Środowisko i Wdrażanie

System jest oparty na środowisku Docker, z podziałem na wyizolowane środowiska przy pomocy dedykowanych plików `docker-compose`:

- **Deweloperskie** (`docker-compose.yml`): Środowisko przeznaczone do codziennej pracy programistycznej. Baza danych jest otwarta na zewnątrz, aby ułatwić debugowanie.
- **Testowe / Staging** (`docker-compose.test.yml`): Środowisko służące do weryfikacji aplikacji przed wdrożeniem. Służy do wykonywania testów jednostkowych oraz wydajnościowych (symulacja obciążenia za pomocą Apache JMeter). Obrazy kontenerów są budowane na stałe (brak hot-reloadu), co gwarantuje wysoką powtarzalność i identyczność testów.
- **Produkcyjne** (`docker-compose.prod.yml`): Docelowe środowisko udostępniane użytkownikom końcowym. Bezpieczeństwo zapewniają ukryte porty bazy danych, wyłączony tryb debugowania oraz klucze API pobierane z bezpiecznych plików `.env`. Baza danych wykorzystuje persistent volumes do trwałego i bezpiecznego przechowywania rzeczywistych informacji.

7 Zapewnienie Jakości

- **Testy jednostkowe:** Zrealizowane za pomocą frameworka Pytest w wyizolowanym środowisku testowym bazy PostgreSQL. Wykorzystują bibliotekę `unittest.mock` do izolacji zapytań HTTP (mockowanie Open-Meteo i Ticketmaster). Zapewniają poprawność zapisów (np. obsługa zjawiska odpięcia sesji `db.refresh()`), działanie logiki cache'u oraz poprawne sprzątanie zasobów testowych.
- **Testy wydajnościowe:** Przeprowadzane przy użyciu Apache JMeter na kontenerze deweloperskim. Mają na celu symulację równoległego ruchu użytkowników na głównym endpointzie aplikacji (`/api/search`). Mierzą przepustowość systemu (Requests Per Second), opóźnienia w odpowiedziach oraz zjawisko "bottleneckingu" na styku FastAPI i bazy danych pod obciążeniem.
- **Monitorowanie i logowanie (Debugowanie):** W aplikacji zaimplementowano scentralizowany system logowania oparty na wbudowanym module `logging`. Rejestruje on kluczowe informacje o przepływie danych (np. przychodzące zapytania, odpytywanie zewnętrznych API) oraz bezpiecznie przechwytuje i loguje błędy (wraz z pełnym stosem wywołań - *Traceback*). Pozwala to na szybkie debugowanie systemu bez ingerencji w kod źródłowy oraz zapobiega awariom serwera w przypadku otrzymania niekompletnych danych od zewnętrznych dostawców.

7.1 Wyniki testów wydajnościowych

Metryka	Wartość
Liczba próbek (Samples)	5000
Przepustowość (Throughput)	49,4 zap/s
Współczynnik błędów	52,62%
Średni czas odp. (Avg)	8827 ms
Min. czas odpowiedzi	0 ms
Maks. czas odpowiedzi	42972 ms

Tabela 1: Wyniki testu przeciążeniowego (Stress Test) dla architektury FastAPI i PostgreSQL

Przeprowadzony test przeciążeniowy (Stress Test) pozwolił zidentyfikować granicę wydajności systemu. Jego wyniki są widoczne w Tabeli 1. W przypadku

Parametr	Load Test (Cold)	Load Test (Warm)
Średni czas	708 ms	591 ms
Maks. czas	9066 ms	5340 ms
Błędy (%)	43.35%	18.90%
Przepustowość	46.0 zap/s	50.4 zap/s

Tabela 2: Wpływ procedury Warm-up na parametry Load Testu

braku zainicjalizowanej pamięci podręcznej system osiągnął maksymalną przepustowość na poziomie 49,4 zapytań na sekundę, po czym nastąpił gwałtowny spadek wydajności, skutkujący współczynnikiem błędów wynoszącym 52,62% (kod 502 Bad Gateway). Główną przyczyną awarii było jednoczesne kierowanie dużej liczby żądań do tych samych zasobów, co doprowadziło do wyczerpania puli połączeń SQLAlchemy oraz blokowania dostępu przez zewnętrzne API Ticketmaster na skutek przekroczenia dopuszczalnej liczby zapytań. Uzyskane wyniki wskazują, że bez zastosowania mechanizmu buforowania system nie jest odporny na nagły i intensywny wzrost ruchu.

Porównanie wyników testów potwierdza istotną rolę pamięci podręcznej. Wyniki zastosowania procedury warm-up widoczne są w Tabeli 2. Polega ona na wcześniejszym zapełnieniu pamięci podręcznej najczęściej wykorzystywanymi danymi, co pozwoliło skrócić średni czas odpowiedzi systemu z 8827 ms do 591 ms, co oznacza poprawę o ponad 93%. Dzięki temu wyeliminowano błędy 502 związane z ograniczeniami narzucanymi przez zewnętrzne API. W efekcie głównym ograniczeniem wydajności stała się infrastruktura aplikacji, w szczególności pojedynczy proces serwera Uvicorn, którego maksymalna przepustowość wynosi około 50 zapytań na sekundę. Pozostałe 18,90% błędów w teście Warm Cache wynikało z wyczerpania puli połączeń do bazy danych przy dużej liczbie jednocześnie obsługiwanych żądań, co wskazuje na potrzebę dalszej optymalizacji konfiguracji puli połączeń.